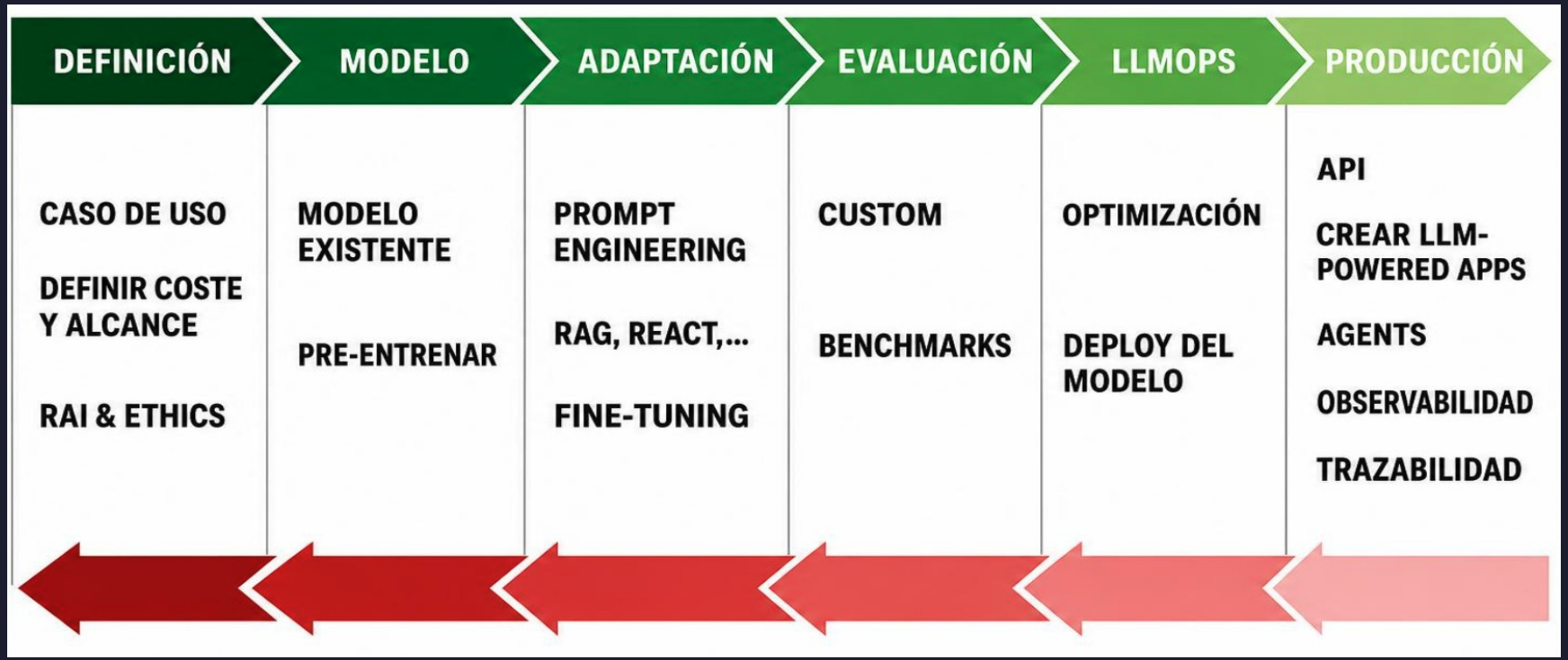
keep coding

AI ENGINEERING

Técnicas modernas en práctica


DEFINICIÓN
MODELO
ADAPTACIÓN
EVALUACIÓN
LLMOPS
PRODUCCIÓN
CASO DE USO
DEFINIR COSTE Y ALCANCE
RAI & ETHICS
MODELO EXISTENTE
PRE-ENTRENAR
PROMPT ENGINEERING
RAG, REACT,...
FINE-TUNING
CUSTOM
BENCHMARKS
OPTIMIZACIÓN
DEPLOY DEL MODELO
API
CREAR LLM-POWERED APPS
AGENTS
OBSERVABILIDAD
TRAZABILIDAD

El problema del full fine-tuning

keep coding

- Caro · entrenas TODOS los pesos del modelo
- Lento · horas o días según el tamaño
- Necesitas mucha GPU y VRAM · inviable en modelos grandes
- Y encima rompe el modelo · olvido catastrófico (Lab 1)
- Por eso nace PEFT/LoRA: entrenar solo una parte

keep coding

LAB 01

Fine-tuning GPT-2

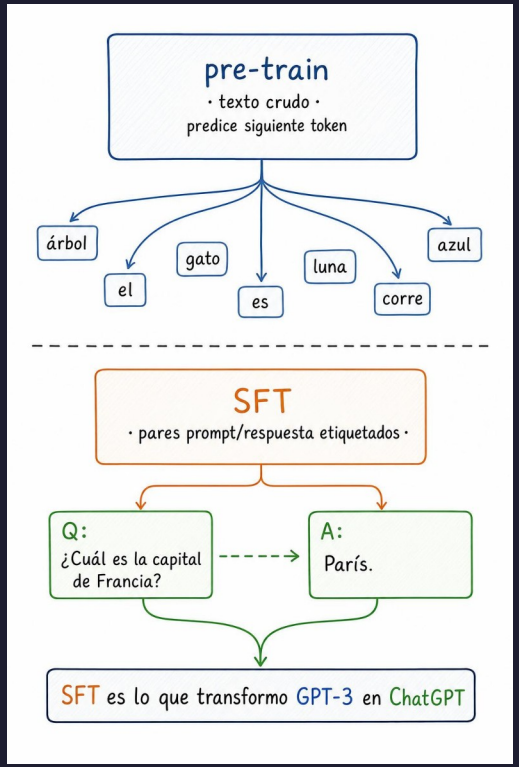
notebooks/01-Fine-tuning GPT-2.ipynb

Full fine-tuning de GPT-2 (124M) para text-to-SQL · mides el olvido catastrófico · T4

SFT · de pre-train a instruct

keep coding

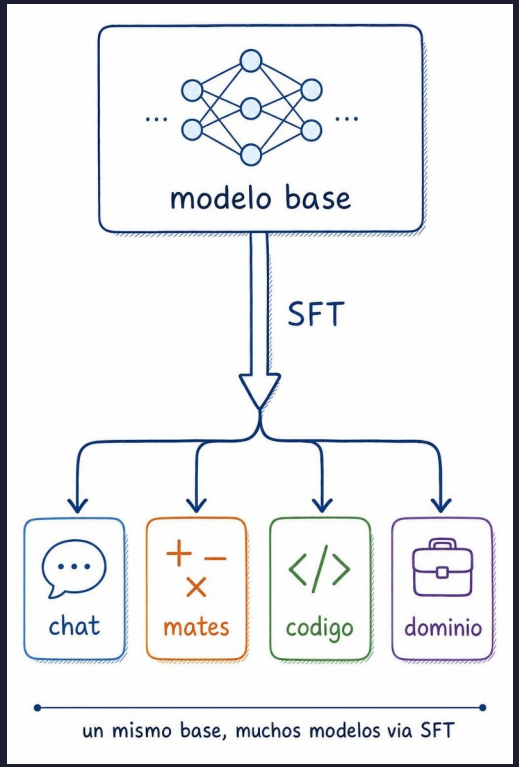
- Pre-train predice el siguiente token sobre texto crudo
- SFT ajusta con pares prompt/respuesta etiquetados
- Es lo que convirtió GPT-3 en algo que sigue instrucciones
- Necesita un dataset etiquetado de calidad



Del modelo base a los especializados

keep coding

- El pre-train deja un modelo BASE: sabe lenguaje, no sigue instrucciones
- Después se pasa por SFT para especializarlo
- Del mismo base salen muchos modelos
- Gemma base -> Gemma-it · Qwen base -> Qwen-Math, Qwen-Coder
- Es el primer paso · luego viene el alignment



PEFT · entrenar solo una parte

keep coding

Congela la mayoría de pesos y entrena un subconjunto minúsculo.

	Fine-tuning completo	PEFT
Cómputo	como entrenar (mucho)	muy poco
Disco	el modelo crece	el adapter es minúsculo
Datos	dataset grande	un dataset pequeño basta
Rendimiento	el mejor	similar, a veces algo peor

Low-Rank Adaptation (LoRA)

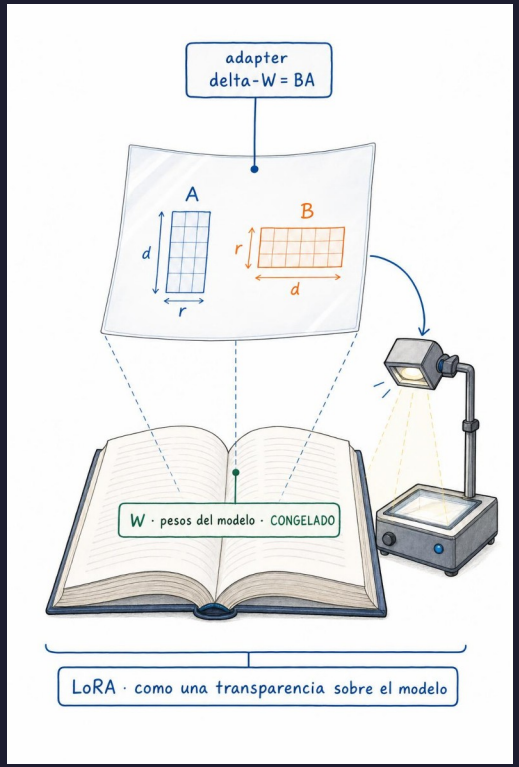
keep coding

- Congela los pesos originales del modelo
- Inyecta matrices pequeñas de bajo rango
- Reduce hasta ~90% los parámetros a entrenar
- Entrenas el adapter, no el modelo entero

LoRA · la transparencia

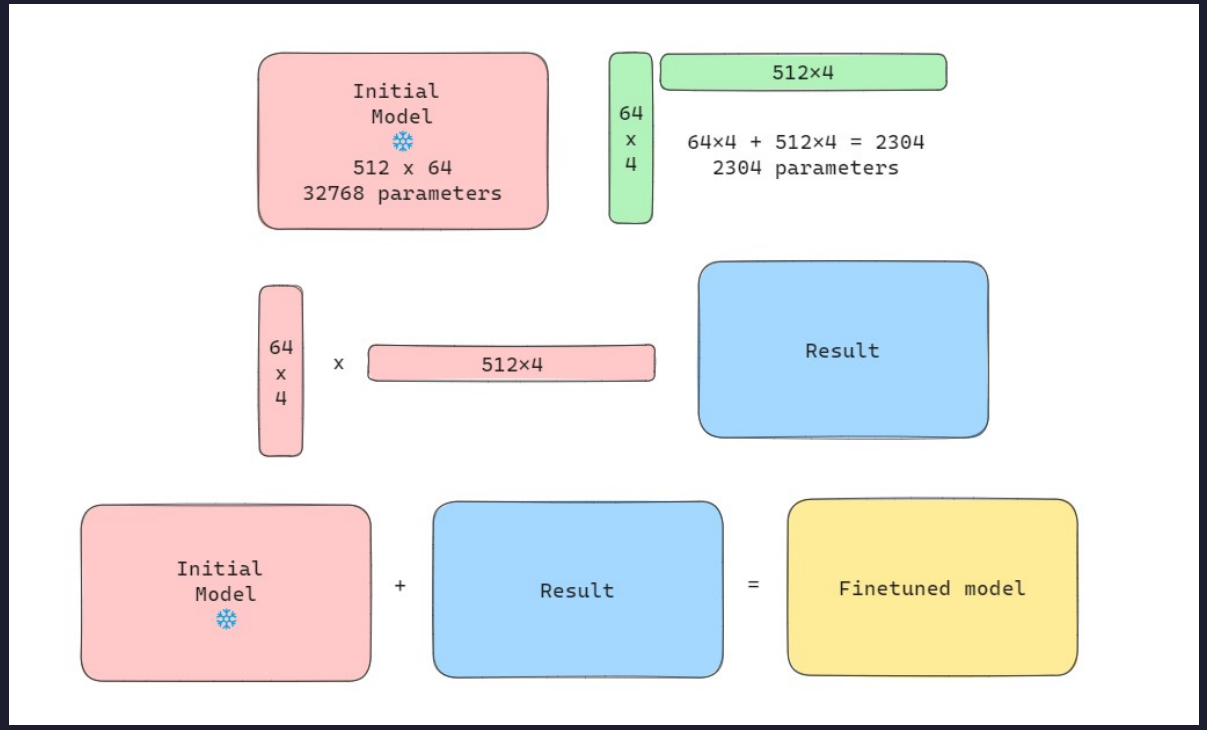
keep coding

- Como una transparencia encima del modelo
- W (base) CONGELADO · solo entrenas A y B
- Forward: h = Wx + BAx
- Quitas el adapter y vuelves al modelo base



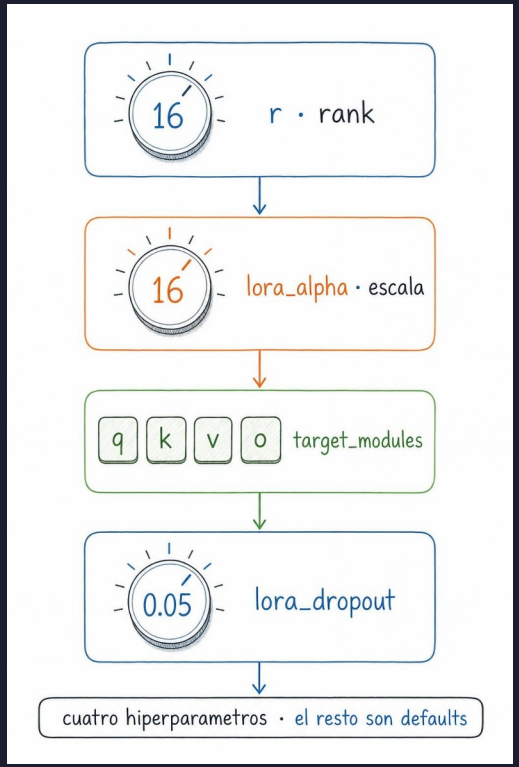
LoRA · matrices pequeñas

W congelado (32.768 params) + B·A de rango 4 (solo 2.304 params) = modelo afinado



LoRA · hiperparámetros clave

- r (rank) · 8/16/32 · capacidad del adapter
- lora_alpha · escala (suele = r o 2r)
- target_modules · q, k, v, o + MLP
- lora_dropout · 0-0.1 (0 por defecto en Unsloth)
- Más r = más capacidad pero más riesgo de overfit



keep coding

LAB 02

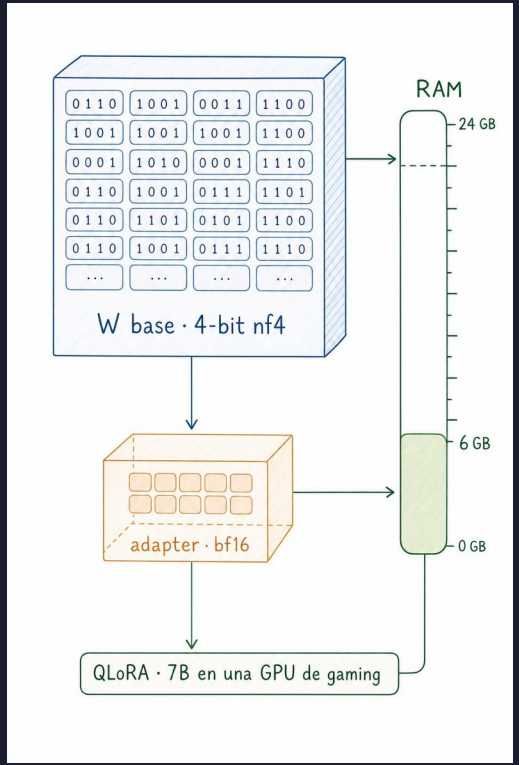
SQL con LoRA

notebooks/02-Fine-tuning SQL con LoRA.ipynb

Misma tarea y evaluación que el Lab 01, con LoRA (peft) · sin olvido · T4

QLoRA · transparencia comprimida

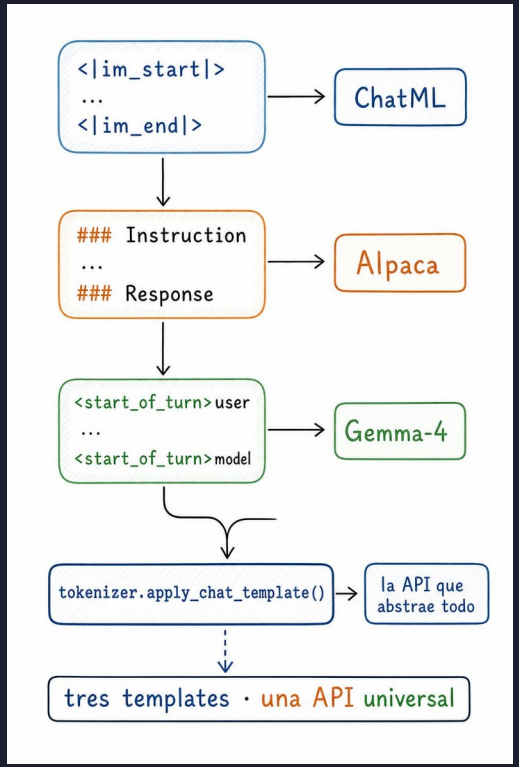
- Base en 4-bit (nf4) + adapter en bf16
- VRAM brutalmente baja · 7B en ~6 GB
- Sin pérdida significativa de calidad
- Permite fine-tunear 13B-65B en una sola GPU



Datasets · chat templates

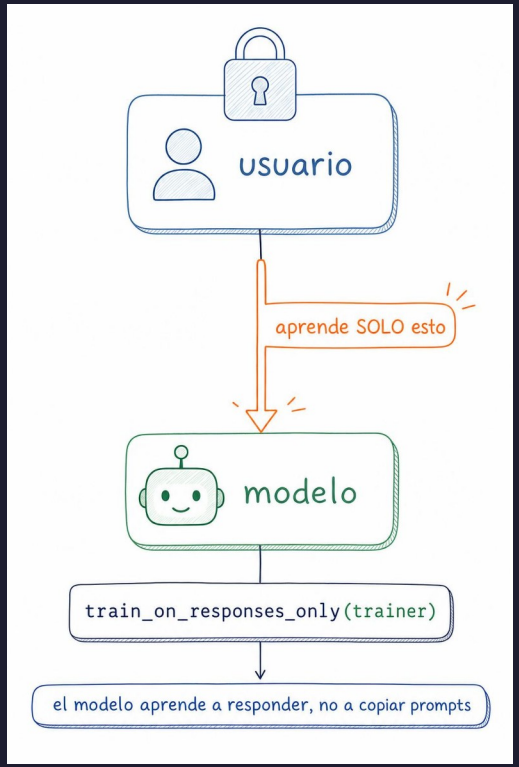
keep coding

- ChatML · <|im_start|>...<|im_end|>
- Alpaca · ### Instruction / ### Response
- Gemma 4 · <|turn>user ... <turn|> (Gemma 3: <start_of_turn>)
- tokenizer.apply_chat_template() lo abstrae



train_on_responses_only

- Enmascara el prompt al calcular la pérdida
- Entrena SOLO sobre las respuestas del assistant
- Evita que el modelo memorice los prompts



Unsloth · el speedup

- 2x más rápido, 70% menos VRAM
- Kernels Triton propios (RoPE, MLP) + packing
- FastModel.from_pretrained() drop-in con HF
- Gemma 4 E4B entrenable en una T4 gratis



Unsloth · requisitos y flujo

keep coding

- VRAM con QLoRA · 7B ~5 GB · 14B ~8.5 GB · 70B ~41 GB
- NVIDIA (CUDA ≥ 7.0, T4 incluida) · AMD e Intel · Mac
- Internamente · PyTorch + Triton + bitsandbytes + TRL
- Flujo · cargar -> QLoRA -> dataset -> train -> eval -> export

Unsloth Studio (marzo 2026)

keep coding

· UI no-code · data prep + train + monitor + export

· 500+ modelos (Llama, Gemma, Qwen, Mistral, Phi)

· Corre local en Mac/Windows/Linux

· Para quien no quiere tocar Python



keep coding

LAB 03

Clasificación de intención

notebooks/03-Clasificacion de intencion con QLoRA.ipynb

QLoRA + Unsloth sobre Gemma 4 E2B/E4B · routing de intención con banking77 · T4

keep coding

LAB 04

Hablar como Milei

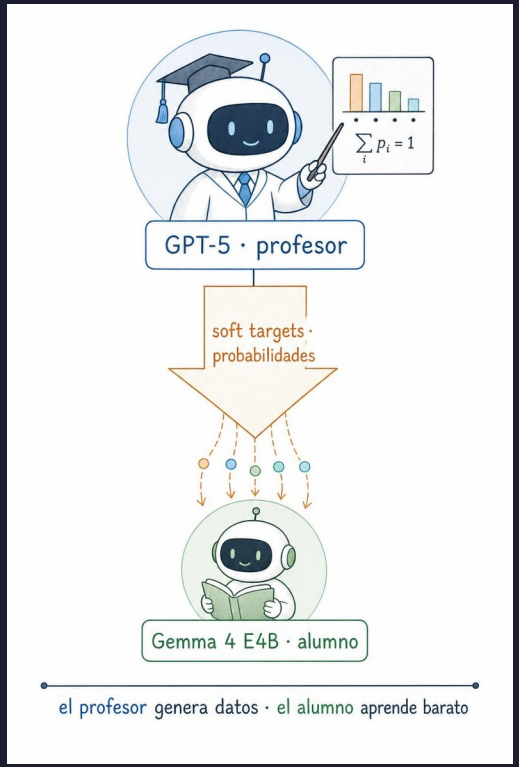
notebooks/04-Hablar como Milei (QLoRA).ipynb

QLoRA + Unsloth sobre Qwen3.5 (2B en T4 · 9B en A100 · 27B en H100) · fine-tuning de estilo

Distillation · teacher -> student

keep coding

- Soft targets (probabilidades) vs hard labels
- Black-box · usar la API del teacher para generar dataset
- White-box · alinear logits (requiere los pesos)
- Teacher caro -> student pequeño y barato en local



keep coding

LAB 05

Distillation con teacher OpenAI

notebooks/05-Distillation teacher OpenAI.ipynb

gpt-5-mini genera 200 Q&A de programación · student Gemma 4 E2B/E4B con QLoRA · OPENAI_API_KEY

Guía de hiperparámetros LoRA (1/2)

keep coding

Hiperparámetro	Qué es	Recomendado
Learning rate	ajuste por paso	2e-4 (inicio) a 5e-6
Epochs	veces que ve el dataset	1 a 3
Rank (r)	dimensiones que entrena	8 a 128 (16-32 equilibrado)
Alpha	escala del adapter	= r o 2r
Dropout	regularización	0 a 0.1 (0 = más rápido)

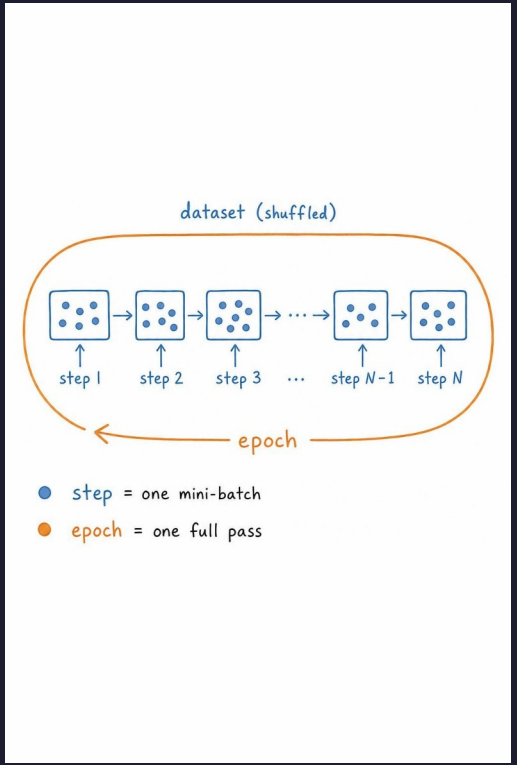
Guía de hiperparámetros LoRA (2/2)

keep coding

Hiperparámetro	Qué es	Recomendado
Batch size	muestras por paso	1 (ctx largo), 2-4 (corto)
Max seq length	tokens por entrada	512 / 1024 / 2048+
Warmup	subida gradual del LR	5-10% de los pasos
Scheduler	cómo cambia el LR	linear o cosine
Weight decay	penaliza pesos grandes	0.01-0.1
Seed	reproducibilidad	3407

Steps vs epochs

- Step · una actualización de pesos sobre un mini-batch
- Epoch · una pasada completa por TODO el dataset
- 1 epoch = (nº ejemplos / batch size) steps
- El entrenamiento se mide en steps, no en epochs
- Pocos datos: piensa en epochs · muchos: en steps



Evitar overfit y underfit

keep coding

- Overfit · accuracy alta en train, baja en real
- -> baja epochs, sube dropout, mezcla dataset general
- Underfit · no aprende ni el train
- -> más epochs, sube rank y alpha, dataset más específico
- Consejo · prueba (LR alto + pocas epochs) vs (LR bajo + más epochs)

Datasets Guide

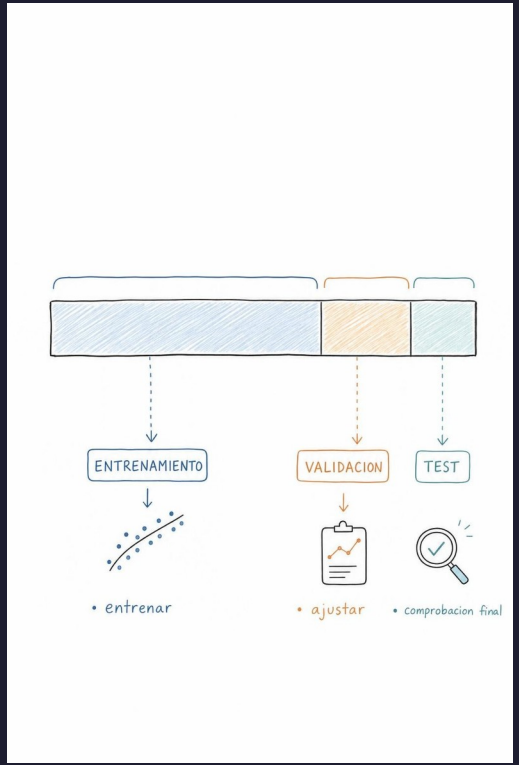
keep coding

El formato manda. Define propósito, idioma y estilo antes de empezar.

Formato	Para qué	Estructura
Raw corpus	pre-entrenamiento continuo	texto continuo
Instruct	tareas (resumir, clasificar)	instruction + output
Conversation	chats multivuelta	ShareGPT / ChatML
RLHF	preferencias	pares con ranking

Train, validación y test

- Train · con lo que el modelo aprende (la mayor parte)
- Validación · ajustar hiperparámetros sin tocar el test
- Test · el examen final · datos que nunca vio
- Nunca evalúes sobre datos de train · te engañas solo
- Split típico · 80 / 10 / 10




keep coding